



Logistic Regression for Machine Learning

by **Jason Brownlee** on [December 6, 2023](#) in **Machine Learning Algorithms** 💬 136

[Share](#)[Share](#)

Post

Logistic regression is another technique borrowed by machine learning from the field of statistics.

It is the go-to method for binary classification problems (problems with two class values). In this post, you will discover the logistic regression algorithm for machine learning.

After reading this post you will know:

- The many names and terms used when describing logistic regression (like log odds and logit).
- The representation used for a logistic regression model.
- Techniques used to learn the coefficients of a logistic regression model from data.
- How to actually make predictions using a learned logistic regression model.
- Where to go for more information if you want to dig a little deeper.
- Problem faced by the algorithm and the latest solution.
- Recent updates in machine learning and deep learning frameworks.
- Logistic regression and XAI.
- Logistic regression and federated learning.

This post was written for developers interested in applied machine learning, specifically predictive modeling. You do not need to have a background in linear algebra or statistics.

Updated Dec 2023:

- Updated existing sections for clarity
- Added section: Major Problem with the Algorithm
- Added section: Updates in Well-known Frameworks for Logistic Regression
- Added section: Logistic Regression: Versatility in Explainable AI and Low-Resource/Federated Environments

Kick-start your project with my new book [Master Machine Learning Algorithms](#), including *step-by-step tutorials* and the *Excel Spreadsheet* files for all examples.

Let's get started.



Learning Algorithm for Logistic Regression
Photo by Michael Vadon, some rights reserved.

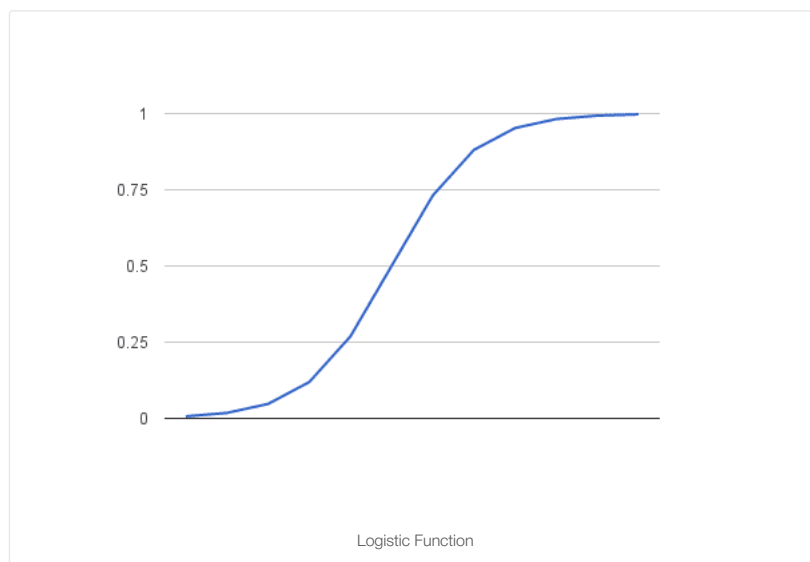
Logistic Function

Logistic regression is named for the function used at the core of the method, the logistic function.

The **logistic function**, also called the sigmoid function was developed by statisticians to describe properties of population growth in ecology, rising quickly and maxing out at the carrying capacity of the environment. It's an S-shaped curve that can take any real-valued number and map it into a value between 0 and 1, but never exactly at those limits.

$$1 / (1 + e^{-\text{value}})$$

Where e is the **base of the natural logarithms** (Euler's number or the EXP() function in your spreadsheet) and value is the actual numerical value that you want to transform. Below is a plot of the numbers between -5 and 5 transformed into the range 0 and 1 using the logistic function.



Now that we know what the logistic function is, let's see how it is used in logistic regression.

Representation Used for Logistic Regression

Logistic regression uses an equation as the representation, very much like linear regression.

Input values (x) are combined linearly using weights or coefficient values (referred to as the Greek capital letter Beta) to predict an output value (y). A key difference from linear regression is that the output value being modeled is a binary value (0 or 1) rather than a numeric value.

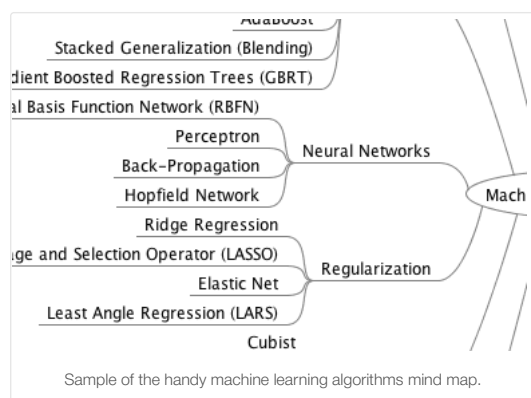
Below is an example logistic regression equation:

$$y = e^{(b_0 + b_1 \cdot x)} / (1 + e^{(b_0 + b_1 \cdot x)})$$

Where y is the predicted output, b₀ is the bias or intercept term and b₁ is the coefficient for the single input value (x). Each column in your input data has an associated b coefficient (a constant real value) that must be learned from your training data.

The actual representation of the model that you would store in memory or in a file are the coefficients in the equation (the beta value or b's).

Get your FREE Algorithms Mind Map



I've created a handy mind map of 60+ algorithms organized by type.

Download it, print it and use it.

[Download For Free](#)

Also get exclusive access to the machine learning algorithms email mini-course.

Logistic Regression Predicts Probabilities (Technical Interlude)

Logistic regression models the probability of the default class (e.g. the first class). For example, if we are modeling people's sex as male or female from their height, then the first class could be male, and the logistic regression model could be written as the probability of male given a person's height, or more formally:

$$P(\text{sex}=\text{male}|\text{height})$$

In contemporary machine learning applications, understanding probability estimation is crucial. Logistic regression predicts probabilities, which are the foundation for classification tasks.

Written another way, we are modeling the probability that an input (X) belongs to the default class (Y=1), and we can write this formally as:

$$P(X) = P(Y=1|X)$$

It's essential to emphasize that logistic regression is not just a classification algorithm; it's a method for estimating probabilities.

Are we predicting probabilities? I thought logistic regression was a classification algorithm.

Logistic regression is a powerful classification technique by estimating the likelihood of an input belonging to a particular class. This estimation is inherently a probability prediction, which must be converted into binary values (0 or 1) to make class predictions. We'll delve deeper into this process shortly when discussing making predictions.

Logistic regression is a linear method, but the predictions are transformed using the logistic function. The impact of this is that we can no longer understand the predictions as a linear combination of the inputs as we can with linear regression. For example, continuing from above, the model can be stated as:

$$p(X) = e^{(b_0 + b_1 * X)} / (1 + e^{(b_0 + b_1 * X)})$$

I don't want to dive into the math too much, but we can turn around the above equation as follows (remember we can remove the e from one side by adding a natural logarithm (ln) to the other):

$$\ln(p(X) / 1 - p(X)) = b_0 + b_1 * X$$

This mathematical transformation allows us to interpret the model more intuitively. The left-hand side represents the log odds or the probit, which is a crucial concept in logistic regression.

This is useful because we can see that the calculation of the output on the right is linear again (just like linear regression), and the input on the left is a log of the probability of the default class.

This ratio on the left is called the odds of the default class (it's historical that we use odds, for example, odds are used in horse racing rather than probabilities). Odds are calculated as a ratio of the probability of the event divided by the probability of not the event, e.g. $0.8/(1-0.8)$ which has the odds of 4. So we could instead write:

$$\ln(\text{odds}) = b_0 + b_1 * X$$

Because the odds are log-transformed, we call this left-hand side the log-odds or the probit. It is possible to use other types of functions for the transform (which is out of scope), but as such, it is common to refer to the transformation that relates the linear regression equation to the probabilities as the link function, e.g., the probit link function.

We can move the exponent back to the right and write it as:

$$\text{odds} = e^{(b_0 + b_1 * X)}$$

All of this provides valuable insights into the inner workings of logistic regression, demonstrating that the model is indeed a linear combination of the inputs. However, this linear combination is related to the log-odds of the default class, making it a powerful tool for probabilistic classification.

Learning the Logistic Regression Model

The coefficients (Beta values b) of the logistic regression algorithm must be estimated from your training data. This is done using maximum-likelihood estimation.

Maximum-likelihood estimation is a common learning algorithm used by a variety of machine learning algorithms, although it does make assumptions about the distribution of your data (more on this when we talk about preparing your data).

The best coefficients would result in a model that would predict a value very close to 1 (e.g. male) for the default class and a value very close to 0 (e.g. female) for the other class. The intuition for maximum-likelihood for logistic regression is that a search procedure seeks values for the coefficients (Beta values) that minimize the error in the probabilities predicted by the model to those in the data (e.g. probability of 1 if the data is the primary class).

We are not going to go into the math of maximum likelihood. It is enough to say that a minimization algorithm is used to optimize the best values for the coefficients for your training data. In practice, the optimization of logistic regression models is often implemented using efficient numerical optimization algorithms such as Limited-memory Broyden–Fletcher–Goldfarb–Shanno (L-BFGS) and Adaptive Moment Estimation (ADAM), which are now considered best practices. These methods provide faster convergence and improved efficiency, particularly when dealing with large datasets.

When you are learning logistic regression, you can implement it yourself from scratch using the much simpler gradient descent algorithm.



Logistic Regression for Machine Learning
Photo by woodleywonderworks, some rights reserved.

Making Predictions with Logistic Regression

Making predictions with a logistic regression model is as simple as plugging in numbers into the logistic regression equation and calculating a result.

Let's make this concrete with a specific example.

Let's say we have a model that can predict whether a person is male or female based on their height (completely fictitious). Given a height of 150cm is the person male or female.

We have learned the coefficients of $b_0 = -100$ and $b_1 = 0.6$. Using the equation above we can calculate the probability of male given a height of 150cm or more formally $P(\text{male}|\text{height}=150)$. We will use $\text{EXP}()$ for e , because that is what you can use if you type this example into your spreadsheet:

$$y = e^{(b_0 + b_1 \cdot X)} / (1 + e^{(b_0 + b_1 \cdot X)})$$

$$y = \exp(-100 + 0.6 \cdot 150) / (1 + \exp(-100 + 0.6 \cdot X))$$

$$y = 0.0000453978687$$

Or a probability of near zero that the person is a male.

In practice we can use the probabilities directly. Because this is classification and we want a crisp answer, we can snap the probabilities to a binary class value, for example:

$$0 \text{ if } p(\text{male}) < 0.5$$

$$1 \text{ if } p(\text{male}) \geq 0.5$$

Now that we know how to make predictions using logistic regression, let's look at how we can prepare our data to get the most from the technique.

Prepare Data for Logistic Regression

The assumptions made by logistic regression about the distribution and relationships in your data are much the same as the assumptions made in linear regression.

Much study has gone into defining these assumptions and precise probabilistic and statistical language is used. My advice is to use these as guidelines or rules of thumb and experiment with different data preparation schemes.

Ultimately in predictive modeling machine learning projects you are laser focused on making accurate predictions rather than interpreting the results. As such, you can break some assumptions as long as the model is robust and performs well.

- **Binary Output Variable:** This might be obvious as we have already mentioned it, but logistic regression is intended for binary (two-class) classification problems. It will predict the probability of an instance belonging to the default class, which can be snapped into a 0 or 1 classification.

- **Remove Noise:** Logistic regression assumes no error in the output variable (y), consider removing outliers and possibly misclassified instances from your training data.
- **Gaussian Distribution:** Logistic regression is a linear algorithm (with a non-linear transform on output). It does assume a linear relationship between the input variables with the output. Data transforms of your input variables that better expose this linear relationship can result in a more accurate model. For example, you can use log, root, Box-Cox and other univariate transforms to better expose this relationship.
- **Remove Correlated Inputs:** Like linear regression, the model can overfit if you have multiple highly-correlated inputs. Consider calculating the pairwise correlations between all inputs and removing highly correlated inputs.
- **Fail to Converge:** It is possible for the expected likelihood estimation process that learns the coefficients to fail to converge. This can happen if there are many highly correlated inputs in your data or the data is very sparse (e.g. lots of zeros in your input data).

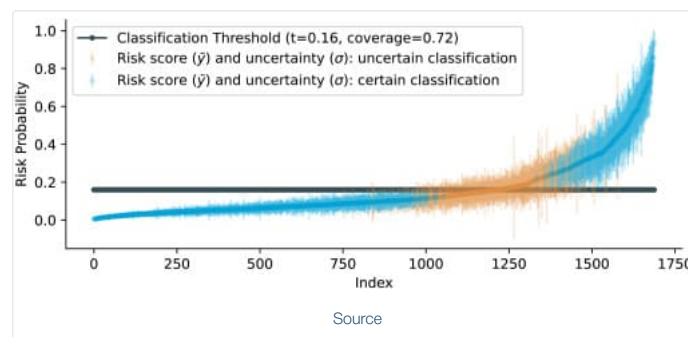
Major Problem with the Algorithm

Skewed class distributions significantly impact the performance of logistic regression, particularly in scenarios where class representation is imbalanced. Logistic regression tends to favor the majority class when one class substantially outweighs the other in instances. Consequently, it loses its ability to identify and predict occurrences in the minority class, resulting in reduced accuracy, precision, and recall, especially affecting the minority class of interest. Consider a medical diagnosis scenario where Logistic Regression is applied to predict a rare disease with only a 3% detection rate. Even if the algorithm predicts all patients as normal, it achieves a seemingly high 97% accuracy while failing to fulfill its actual purpose.

Moreover, adversarial attacks disrupt logistic regression's performance by introducing subtle input data changes. These alterations can lead to erroneous predictions, diminishing model accuracy. Logistic regression's vulnerability to minor input perturbations makes it susceptible to such attacks, compromising reliability, particularly in security-sensitive applications. Consider the example of spam email classification where you are using a logistic regression algorithm under the hood. Logistic regression, which is like an email filter, is vulnerable to subtle manipulations made by cunning spammers in their messages. These small alterations can confuse the filter, causing it to misclassify spam as legitimate emails and vice versa. As a consequence, the reliability of logistic regression in email filtering is compromised, emphasizing the necessity for advanced defenses and countermeasures against such threats.

Latest Approach to the Algorithm (2023)

Additionally, the paper “A Bayesian approach to predictive uncertainty” (2023) introduced a Bayesian approach to logistic regression, allowing for accurate classification along with quantifiable uncertainty estimates for each prediction. This study applied Bayesian logistic LASSO regression (BLLR) to predict acute care utilization (ACU) risk in cancer patients commencing chemotherapy using real-world electronic health record (EHR) data from over 8,000 patients. Bayesian logistic LASSO regression (BLLR) models outperform standard logistic LASSO regression, delivering superior predictive accuracy, well-calibrated estimations, and more informative uncertainty assessments.



The figure above shows sorted final risk predictions (mean of the predictive distribution, $\hat{y}$) with uncertainty range (standard deviation, $\pm\sigma$). The predictions whose uncertainty does not exceed the decision threshold (certain classifications) are coloured blue, and those that do (uncertain classifications) are coloured orange. The dark gray line is the chosen classification threshold by researchers, and is at 0.16, the event rate. The ratio of certain predictions (coverage) with the Bayesian model turned out to be **0.72**.

Updates in Well-known Frameworks for Logistic Regression

Notable updates have been observed in specialized libraries, most of which have focused on improved efficiency for the algorithm.

- **TensorFlow 2.0:** TensorFlow 2.0 and later versions have introduced an eager execution mode, which simplifies logistic regression model development and debugging.
- **Scikit-learn:** Scikit-learn now offers improved support for handling multi-class problems, making logistic regression more versatile in tackling complex classification tasks.
- **Statsmodels:** Statsmodels, another popular library, has enhanced its capabilities for statistical analysis and hypothesis testing, which can be invaluable for logistic regression-based inferential modeling.
- **InterpretML:** Tools like the InterpretML library now empower users to gain deeper insights and interpret the outcomes of logistic regression models, further enhancing their utility and interpretability.

Machine learning frameworks like PyTorch Lightning and TensorFlow Serving offer streamlined solutions for training and deploying logistic regression models, optimizing efficiency and scalability.

Logistic Regression: Versatility in Explainable AI and Low-Resource/Federated Environments

Logistic regression's adaptability extends to the burgeoning field of Explainable AI (XAI), where interpretable models are essential for understanding and justifying AI-driven decisions. Logistic regression's simplicity and clear parameter interpretations make it a valuable tool for creating transparent and interpretable models. Its ability to provide intuitive insights into feature importance and model output is highly advantageous in applications where transparency and trustworthiness are paramount, such as healthcare, finance, and legal domains.

Furthermore, logistic regression plays a vital role in low-resource and federated settings. In scenarios where data is limited or distributed across multiple sources, logistic regression's lightweight computational requirements and efficient training make it an ideal choice. Its ability to offer reliable predictions while maintaining data privacy and preserving resource efficiency has positioned logistic regression as a foundational technique in addressing challenges in low-resource and federated learning environments.

Further Reading

There is a lot of material available on logistic regression. It is a favorite in many disciplines such as life sciences and economics.

Logistic Regression Resources

Checkout some of the books below for more details on the logistic regression algorithm.

- [Generalized Linear Models](#)
- [Logistic Regression: A Primer](#)
- [Applied Logistic Regression](#)
- [Logistic Regression: A Self-Learning Text \[PDF\]](#).

Logistic Regression in Machine Learning

For a machine learning focus (e.g. on making accurate predictions only), take a look at the coverage of logistic regression in some of the popular machine learning texts below:

- [Artificial Intelligence: A Modern Approach](#), pages 725-727
- [Machine Learning for Hackers](#), pages 178-182
- [An Introduction to Statistical Learning: with Applications in R](#), pages 130-137
- [The Elements of Statistical Learning: Data Mining, Inference, and Prediction](#), pages 119-128
- [Applied Predictive Modeling](#), pages 282-287

If I were to pick one, I'd point to [An Introduction to Statistical Learning](#). It's an excellent book all round.

Summary

In this post you discovered the logistic regression algorithm for machine learning and predictive modeling. You covered a lot of ground and learned:

- What the logistic function is and how it is used in logistic regression.
- That the key representation in logistic regression are the coefficients, just like linear regression.
- That the coefficients in logistic regression are estimated using a process called maximum-likelihood estimation.
- Making predictions using logistic regression is so easy that you can do it in excel.
- That the data preparation for logistic regression is much like linear regression.
- Problem faced by the algorithm and latest solution
- Recent updates in machine learning and deep learning frameworks
- Logistic regression and XAI
- Logistic regression and federated learning

Do you have any questions about logistic regression or about this post?

Leave a comment and ask, I will do my best to answer.

Discover How Machine Learning Algorithms Work!

See How Algorithms Work in Minutes

...with just arithmetic and simple examples

Discover how in my new Ebook:
[Master Machine Learning Algorithms](#)

It covers **explanations** and **examples** of **10 top algorithms**, like:
Linear Regression, k-Nearest Neighbors, Support Vector Machines and much more...